

AFYANOVA V3 – ENTERPRISE HIS GAP CLOSURE

Frontend Audit

A route-by-route and file-by-file review of the Vue 3 / Inertia frontend — reachability against every backend endpoint, security patterns, error handling, and the health of the surrounding tooling. Companion to the backend audit that opened this engagement.

Scope	Volume	Routes checked	Unreferenced
94 Vue files	~19,000 lines	120 named	31 routes
Method	Static + manual read		

4

Critical findings

4

Medium findings

4

Low findings

6

Things holding up well

Critical 4 findings

01 UNREACHABLE FEATURE

The 2FA settings page is completely unreachable through the UI

TwoFactorAuthentication.vue exists, renders, and is fully covered by backend tests — but nothing in the app's navigation, profile menu, or layout links to it. A user has no way to discover or enable MFA short of typing the URL from memory. For a security feature this central, that's effectively "doesn't exist" from a real user's perspective.

```
grep -rn "TwoFactorAuthentication\b|two-factor" Layouts/*.vue
Pages/Profile/*.vue Components/Workspace/*.vue
→ zero matches across the entire app
```

02 DATA HANDLING INCONSISTENCY

Patient registration drafts plaintext PII — including the national ID — to sessionStorage

A "don't lose your work on power cut" autosave writes the full registration form to sessionStorage on every keystroke: name, DOB, phone, address, and the complete 20-digit NIDA number. This is the exact field the backend encrypts at rest with AES-256-GCM. Session-scoped storage isn't persistent or cross-origin, so this is moderate in practice — but it's a real inconsistency in how seriously the two layers treat the same data, and it's readable by anyone with devtools access to an unlocked, mid-session workstation.

```
Pages/Domains/Patient/Register.vue
saveDraftToSession() → sessionStorage.setItem(DRAFT_STORAGE_KEY,
JSON.stringify({ ...nida, ...phone, ...street_address }))
watch() fires on every keystroke in first_name, last_name, dob, phone, nida,
street_address
```

03

PROCESS GAP

CI has never actually run against this code

The GitHub Actions workflow is well-built — it provisions a real Postgres service container and points Pest at it, which means it would have caught the migration-ordering and RLS bugs found by hand earlier in this engagement. But the project was never made a git repository, so no commit has ever triggered it. Every green result reported so far — 166/166 tests, clean Pint, clean `migrate:fresh` — came from local runs and manual Postgres spot-checks, never the pipeline itself firing.

```
.github/workflows/ci.yml
- postgres:16 service container, DB_CONNECTION: postgres for the Pest step
- phpunit.xml has no force="true" on <env name="DB_CONNECTION">, so CI's
exported postgres env wins
- repository has no .git — confirmed at session start ("Is a git
repository: false")
```

04

COVERAGE GAP

Zero frontend UI for all 8 Phase 2/3 clinical modules

Consent, referral, immunization, ANC visits, partograph entries, the problem list, medication reconciliation, and radiology order/sign/amend — every one of these backend modules is complete, authorized, and tested, and every one is unreachable from the browser. Confirmed by cross-referencing all 120 named routes against every `route()` call in the frontend.

```
clinical.consent.store · clinical.referral.store ·
clinical.immunization.store
```

```
clinical.anc.store · clinical.partograph.store ·  
clinical.problems.store/.resolve  
pharmacy.reconciliation.store · radiology.orders.store ·  
radiology.report.sign/.amend  
→ 0 references anywhere under resources/js
```

Medium 4 findings

05 COVERAGE GAP

Orphaned routes beyond the new modules

The same route cross-reference surfaced gaps predating this engagement's Phase 2/3 work — existing features with a backend endpoint and no way to trigger it from the UI that already renders around it.

```
pharmacy.inventory          — page exists, zero navigation path to it  
access-control.facilities.assign — no UI trigger in  
AccessControlWorkspace.vue  
appointments.store          — can check a patient in, cannot create an  
appointment  
billing.generate / .invoices.adjust / .invoices.issue  
inventory.catalog.search / .dda-logs.store / .stocktake.store  
                             — despite InventoryWorkspace.vue already at 2,066  
lines
```

06 UX / CLINICAL SAFETY

Two of the largest workspaces have no field-level error handling

InventoryWorkspace.vue (7 separate forms) and AccessControlWorkspace.vue (2 forms) never read `form.errors` and never register an `onError` callback. A validation failure falls back to a single generic "Validation error occurred" banner from the shared layout — with no indication of which field, or which of several forms on the page, actually failed.

```
Layouts/AfyaShell.vue  
flashError = page.props.flash?.error || (errors present ? 'Validation error  
occurred' : null)  
→ the only feedback path for requisitionForm, newItemForm, transferForm,  
poForm,  
grnForm, confirmTransferForm, assignRoleForm, rolePermissionsForm
```

Two parallel route sets for lab specimen collection

`lab-orders.collect / lab-orders.results` (on `LabOrderController`) sit alongside `laboratory.items.collect / laboratory.items.results` (on `LaboratoryWorkspaceController`) — functionally the same action. Only the second pair is ever called from the UI; the first looks like it was superseded in an earlier phase and never removed.

08 INCOMPLETE FLOW

No way to manually end a break-glass session early

`clinical.break-glass.destroy` is wired on the backend but never called from the frontend. Not a security issue — the override still auto-expires in 5 minutes regardless — but there's no way to close it out the moment the emergency resolves.

Low 4 findings

09 ACCESSIBILITY

Sparse ARIA coverage

Only 15 of 94 Vue files use any `aria-*` attribute. Worth improving incrementally in a clinical tool used under time pressure, rather than as a one-off sweep.

10 ROADMAP, CONFIRMED

No i18n infrastructure at all

No `vue-i18n`, no `$t()` calls, nothing — the entire UI is hardcoded English. This isn't a new discovery so much as confirmation: Phase 5's "Swahili i18n" line item is genuinely unstarted, not partially done.

11 TEST COVERAGE

No frontend test coverage whatsoever

No Vitest, Jest, Cypress, or Playwright in `package.json`; no test files anywhere under `resources/js`. For a ~19,000-line application moving PII and money, this is a real gap — the backend's 166 Pest tests have no frontend counterpart at any level.

No TypeScript, no ESLint, no Prettier

Plain JavaScript throughout, no linter or formatter configuration. Nothing catches a typo'd prop name or an inconsistent style before it ships.

Holding up well 6 things worth keeping as-is

- + **No XSS surface found.** Zero `v-html` usage anywhere across all 94 files.
- + **Clean dependency tree.** `npm audit` reports 0 vulnerabilities across both production and dev dependencies.
- + **No secrets, no debug debris.** Zero hardcoded API keys/tokens, zero stray `console.log` or `debugger` statements.
- + **Double-submission is a non-issue.** All 18 forms using `useForm()` correctly gate their submit button on `form.processing` — including the payment and refund flows, which is exactly where that would have hurt.
- + **The MFA and break-glass frontends that do exist are well-built.** Correct field names matching the backend contract, mirrored validation (20-character justification counter, 6-digit code masking), honest UX around the 5-minute expiry and audit trail.
- + **This is not a prototype.** A consistent, substantial design system (Afya components + shadcn-style primitives) reused across every workspace — 12 fully-built pages averaging over 1,000 lines each.

METHODOLOGY

- Extracted all 120 named backend routes via `artisan route:list`, extracted every